Software Requirements Specification for Impossible Chess: Pentachess

Version 1

Team 6
Mian Rafay
Karl Sader
Saad Tariq
Emaan Khan
Brandon Bosman
October 11th, 2024, 11:59 PM

Table of Contents

Team Member Contributions **3**
Project Personnel **3**
1. Purpose of the Project **3**
2. Stakeholders **3**
2.1 Primary Stakeholders 3
2.2 Secondary Stakeholders 3
2.3 Tertiary Stakeholders 4
3. Mandated Constraints **4**
3.1 Solution Constraints 4
3.2 Implementation Environment of the Current System 4
3.3 Anticipated Workplace 4
3.4 Schedule Constraints 4
3.5 Budget Constraints 4
4. Glossary of All Terms, Including Acronyms, Used by Stakeholders involved in the Project **4**
5. Relevant Facts And Assumptions **4**
5.1 Relevant Facts 4
5.2 Business Rules 6
5.3 Assumptions 6
6. The Scope of the Work **7**
6.1 The Context of the Work 7
6.2 Work Partitioning 7
6.3 Specifying a Business Use Case (BUC) 7
7. The Scope of the Product **8**
7.1 Product Boundary 8
7.2 Product Use Case Table 8
7.3 Individual Product Use Cases (PUC's) 9
8. Functional Requirements **9**
9. Look and Feel Requirements **11**
9.1 Appearance Requirements 11
9.2 Style Requirements 11
10. Usability and Humanity Requirements **11**
10.1 Ease of Use Requirements 11
10.2 Personalization and Internationalization Requirements 11
10.3 Learning Requirements 12
10.4 Understandability and Politeness Requirements 12
10.5 Accessibility Requirements 12
11. Performance Requirements **12**
11.1 Speed and Latency Requirements 12
11.2 Precision or Accuracy Requirements 12

11.3 Robustness or Fault-Tolerance Requirements 12
11.4 Capacity Requirements 12
11.5 Scalability or Extensibility Requirements 12
11.6 Longevity Requirements 12
12. Operational and Environmental Requirements 13
12.1 Expected Physical Environment 13
12.2 Requirements for Interfacing with Adjacent Systems 13
12.3 Productization Requirements 13
12.4 Release Requirements 13
13. Maintainability and Support Requirements 13
13.1 Maintenance Requirements 13
13.2 Supportability Requirements 13
13.3 Adaptability Requirements 13
14. Security Requirements 14
14.1 Access Requirements 14
14.2 Privacy Requirements 14
14.3 Immunity Requirements 14
15. Compliance Requirements 14
15.1 Legal Requirements 14
15.2 Standards Compliance Requirements 14
16. Open Issues 14
17. Off-the-Shelf Solutions 15
17.1 Ready-Made Products 15
17.2 Reusable Components 15
17.3 Products That Can Be Copied 15
18. Costs 15
19. User Documentation and Training 15
19.1 User Documentation Requirements 15
19.2 Training Requirements 15
20. Waiting Room 15

Team Member Contributions

Sections Worked On	Team Member	Email
1, 2, 3.1, 3.4, 3.5, 10.1-10.5, 13.1-13.3	Brandon Bosman	bosmanb@mcmaster.ca
8, 9, 11, 12.1, 14, 16	Mian Rafay	rafaym1@mcmaster.ca
3.2, 3.3, 5.3, 16, 17, 18, 19, 20	Saad Tariq	tariqs26@mcmaster.ca
7.1, 7.2, 7.3, 15.1, 15.2 (proj plan)	Karl Sader	saderk@mcmaster.ca
5.1, 5.2, 6.1, 6.2, 6.3, 12.2, 12.3, 12.4	Emaan Khan	khanm345@mcmaster.ca

Project Personnel

Team Member(s)	Roles
Brandon Bosman	Team Manager, various code support
Mian Rafay, Karl Sader	Backend Development
Saad Tariq, Emaan Khan	Frontend Development
All team members	Final Reviews

1. Purpose of the Project

The purpose of this project is to make Pentachess, a challenging and interesting twist on the original game of chess, more accessible to the public. Because of the complexity of the rules, it would be very difficult to play as a board game, but we believe it is the perfect application for a computer game. Not only will a computer game automate difficult rules and present them in a clean GUI, but it will also allow for new ways to play, such as online matches.

2. Stakeholders

2.1 Primary Stakeholders

Our primary stakeholders will be players. While we expect a large portion of these to have a prior interest in chess, we also hope to attract non-chess players. We will work with them to understand their needs and desires and listen to their feedback as we work on the product.

Another primary stakeholder is Dr. Paul Rapoport, the creator of the game and supervisor of the project. We will work under him, ensuring our work matches the rules of his game and presents it suitably.

2.2 Secondary Stakeholders

The development team is another stakeholder. We will be involved with the day-to-day work on the product and the documentation surrounding it. We will have meetings with ourselves and other stakeholders to ensure everyone is on the same page and stays on task.

2.3 Tertiary Stakeholders

The greater chess community is another stakeholder, though to a smaller degree. We hope this game, in a small way, encourages people to engage with chess and the chess community. Another stakeholder is the large community of players of chess variants, of which there are many.

3. Mandated Constraints

3.1 Solution Constraints

The solution must show the board of the game, with its 90 pentagonal pieces. It must include 18 pieces on each side, for a total of 36 pieces. Each piece has its own rules for how pieces may move and capture, and the solution must accurately follow those rules during gameplay. The solution must also give an explanation to players of the overarching rules, to explain why a piece may move where it may move. Lastly, the solution must involve online connectivity, to allow users to play games via online multiplayer.

3.2 Implementation Environment of the Current System

The system consists of three main components: the client, server, and database. The client is a web application through which users interact with the system. The server will manage WebSocket connections to facilitate real-time updates between the client and server and handle API routes for authentication and retrieving and modifying data. The database will store all relevant user and game data.

3.3 Anticipated Workplace

Hybrid (online/McMaster University)

3.4 Schedule Constraints

The solution must be completed within the timeframe set by the course. That is, it must be fully completed, with a working website deployed, by April 4, 2025. This gives the team slightly over 6 months to create it, from start to finish.

3.5 Budget Constraints

As this is not intended to be a commercial venture but simply a class project, the solution must minimize monetary costs.

4. Glossary of All Terms, Including Acronyms, Used by Stakeholders involved in the Project

Term	Definition
GUI	Graphical User Interface.
OAuth	OAuth (Open Authorization) is an open standard for access delegation, commonly used to grant websites or applications limited access to user information from other platforms such as Google without exposing passwords.

5. Relevant Facts And Assumptions

5.1 Relevant Facts

The board consists of three decagons (90 cells in total) with an outer decagon (containing 50 cells), a middle decagon (containing 30 cells), and an inner decagon (containing 10 cells). The primary objective is similar to classical chess; the goal is to checkmate the opposing player's king. The player with pieces of the lighter colour moves first and will alternate turns afterward only being

able to move a single piece in each turn. Also, in each turn, there will be a set time when a player must make a move and five seconds will be added to each player's timer after each turn. The following are the rules for movement, capturing, and promotion for each piece:

Piece	Movement, Capturing, and Promotion Rules

Rook	**Movement:**
	• Can move around the current decagon across edges, however, it cannot return to its starting cell (no 360° turn).
	• Can switch decagons by moving across an edge, however, in one move when moving:
	  ◦ 20 of the 50 outer cells to the middle decagon.
	  ◦ 20 of the 30 middle cells to the outer cells to the outer decagon.
	  ◦ The other 10 of the 30 middle cells to the inner decagon.
	  ◦ All 10 of the inner cells to the middle decagon.
	Capture:
	• Can capture as it moves.

Bishop	**Movement:** • Can move around the current decagon across vertices, however, it must remain in its colour and it cannot return to its starting cell (no 360° turn). • Can switch decagons by moving across a vertex, however, it can only move in a straight line to a cell with the same orientation (it enables moving two decagons in some instances). **Capture:** • Can capture as it moves.
King	**Movement:** • Can move as a rook or bishop, however, by only one step. **Capture:** • Can capture as a rook or bishop, however, by only one step.
Queen	**Movement:** • Can move as a rook or bishop. **Capture:** • Can capture as a rook or bishop.
Knight	**Movement:** • Can move one step across a different colour vertex, excluding moves by a single-step rook. **Capture:** • Can capture one step across a different colour vertex, excluding captures by a single-step rook.

Pawn	**Movement:**
	• Can move one step in the current decagon in the direction indicated by its colour, and it can move across an edge to a cell of a different colour:
	  ◦ Red or dark green: clockwise.
	  ◦ Pink of light green: counterclockwise.
	• Can move one step and switch to a different decagon by crossing an edge

	(regardless of direction). • Can initially move two steps in its current decagon. **Capture:** • Can capture one step in the current decagon in the direction indicated by its colour, and it can capture across a vertex to a cell of the same colour. • Can capture one step and switch to a different decagon by crossing a vertex (regardless of direction). • Can capture two targets in different decagons. • Cannot initially capture two steps in its current decagon. **Promotion**: • Can be promoted to any piece but itself or a king if it is a red or pink pawn and it has reached a cell between 26 and 33. • Can be promoted to any piece but itself or a king if it is a light or dark green pawn and it has reached a cell between 1 and 8. **Note:** • There is no castling or en passant capture.
Berolina's Pawn	**Movement:** • Can move as a pawn captures. • Cannot initially move two steps in its current decagon. **Capture:** • Can capture as a pawn moves. **Promotion:** • Cannot be promoted.

5.2 Business Rules

1. Pieces must comply with all the rules of the game, including, movement, promotion, and capturing rules
2. In each turn, all possible moves for a selected piece must be shown with the ability to drag and drop pieces to their correct position
3. During a game, all captured pieces must be displayed for the player
4. Users can only move a single piece in each term within the specified time limit
5. After each move made by a user, the application must check for conditions such as stalemate or checkmate to determine if the player has won or there has been a draw
6. The application must be user-friendly to all types of users (novice and experienced)
7. There must be easy-to-access instructions for users to refer to regarding how to play Pentachess and navigate the website

5.3 Assumptions

1. Users will access the application using a modern web browser such as Chrome, Firefox, Microsoft Edge, or Safari.
2. Users will have a stable internet connection throughout their online session.
3. Users are expected to have basic proficiency in navigating web applications.
4. The system will be accessed primarily on desktop or laptop devices, though it may support mobile platforms.

6. The Scope of the Work

6.1 The Context of the Work

Pentachess is a full-stack web application that provides chess players with an amazing opportunity to reignite their passion for the game with a fascinating twist on the standard version of chess. It will allow users to play 1-on-1 games online in real-time with each player allocated 18 pieces. Each piece type moves and interacts based on specific rules, which will be provided to each player. When it is either player's turn, they can view all possible movements of the piece they click on. Once a piece is selected, all pentagons where the piece can legally move are highlighted. Additionally, there will be an implemented timer which can be set based on the initial game setup. This timer will ensure there are no idle players.

As mentioned previously, we will be building a full-stack web application. With this in mind, our project will consist of programming languages best suited for web applications. These include HTML, CSS, JavaScript (React), and/or Kotlin. For the backend, we will need to use frameworks and libraries for creating REST APIs (node.js, express.js), managing web socket connections (socket.io), and ORMs (Prisma) for interfacing with the database which will either be SQL (PostgreSQL, MySQL) or NoSQL (MongoDB) based. Furthermore, the stakeholders involved in this project will be chess players, the project supervisor (Dr. Paul Rapoport), and those working on the project directly such as designers, programmers, and testers.

6.2 Work Partitioning

The workload and responsibilities within this project have been divided so that each member's strengths are utilized while ensuring members can learn different aspects of full-stack web development.

- **Frontend Development**: Design and build the user interface (piece movements, user interactions, and Pentachess board)
    - **Members**: Emaan Khan and Saad Tariq
- **Backend Development**: Implement the rules and logic of the game, integrate the database, implement the server
    - **Members**: Mian Rafay and Karl Sader
- **Project Management**: Contribute to both frontend and backend development, coordinate group meetings, and oversee progress and goals within the project
    - **Members**: Brandon Bosman
- **Final Reviews**: Ensure code quality and functionality meet the project's standards and objectives
    - **Members**: Emaan Khan, Saad Tariq, Mian Rafay, Karl Sader and Dr. Paul Rapoport

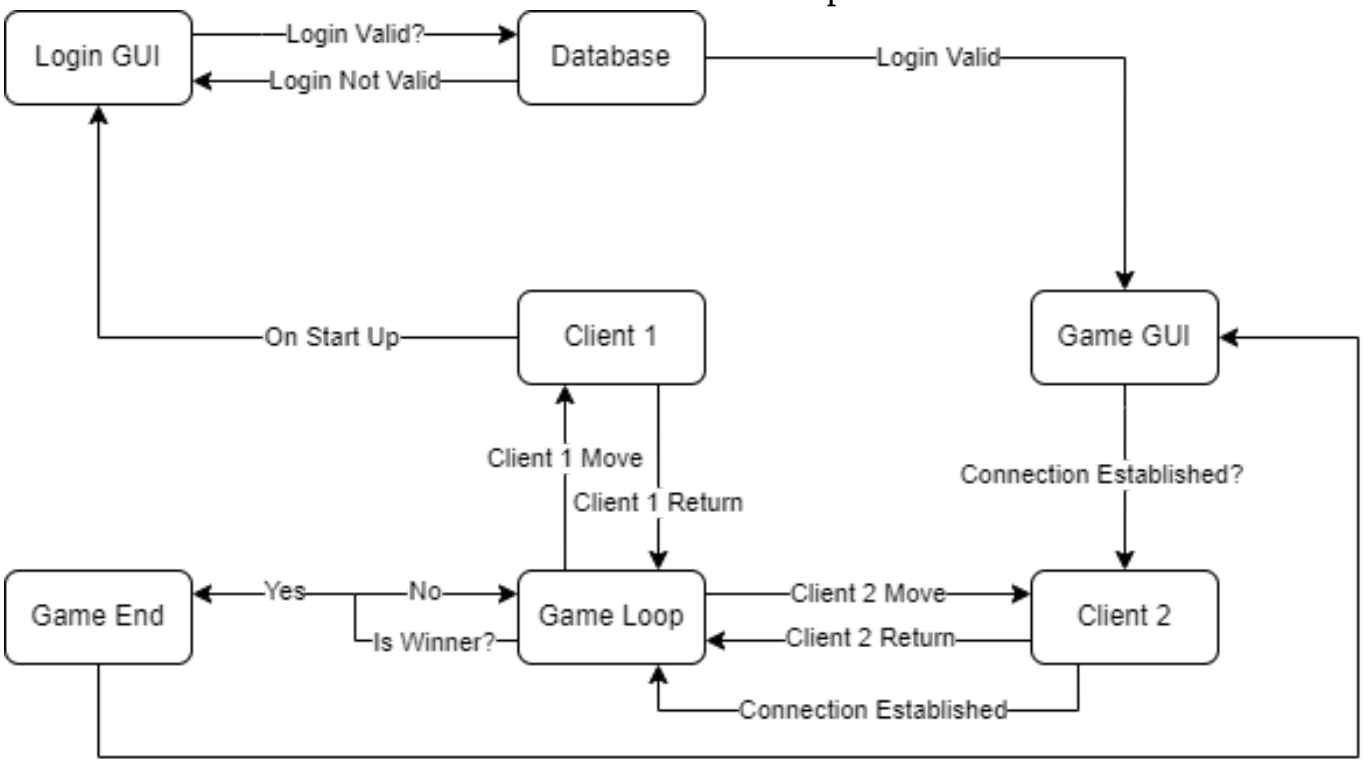
6.3 Specifying a Business Use Case (BUC)

The primary business use case for the Pentachess application is to enable users to engage in a stimulating mental challenge while also enjoying themselves. It offers a multiplayer component where users can engage in matches with each other via a web browser. The application captures the movement and interaction of pieces, upholds game rules, and manages all in-game progress. The project aims to foster a strong community of players by providing an engaging method of playing.

7. The Scope of the Product

7.1 Product Boundary

Below is a state-like diagram to represent the flow of our application. The starting point is 'Client 1' in the center and the first transition it takes is 'On Start Up':



7.2 Product Use Case Table

Below is a table describing each of the high-level states in the diagram displayed in section 7.1 above:

State	Description
Client 1	Client 1 is the first client who opens the web application. Think of this as 'you'.
Login GUI	The Login GUI is an important GUI to verify and validate a user's identity.
Database	The database will store all important data, consisting of, most importantly, login information and game state.
Game GUI	The Game GUI is the most important screen for the player. It displays the board, all its pieces, and where the game will be played.
Client 2	Client 2 is the second client who goes through all the same steps as Client 1 and tries to connect to Client 1 so the game can be played.
Game Loop	The Game Loop is the core of the game consisting of moves between the two players.
Game End	The Game End decides the winner of the game through a checkmate (or the draw of a game through a stalemate).

7.3 Individual Product Use Cases (PUC's)

In this section, I'll briefly describe the process flow of the high-level states given in the diagram and table in sections 7.1 and 7.2 above:
Client 1 is the starting state. On start-up, the **Login GUI** will be displayed where the user needs to input valid logic information. This information will be checked in a **Database**. If the information is invalid, the user will have to re-enter login info until valid info has been entered. Once valid login info has been entered, the **Game GUI** will be displayed to the user. From here, a connection can be established with another player, **Client 2**. After a connection has been properly established, the **Game Loop** will begin where **Client 1** and **Client 2** will alternately enter moves until a winner is decided. When a winner is determined, the game will end (**Game End**), and both users will be sent back to the original **Game GUI**.

8. Functional Requirements

* **p0** (highest priority) and **p3** (lowest priority)

Frontend Requirements	**Priority**
User Account Management	
Ability to register for an account.	**p0**
Ability to update login credentials and account information.	**p2**
Option for password recovery.	**p0**
Game Setup	
Ability to start a local game.	**p0**
Ability to start an online game.	**p0**
Ability to join an online game (random search or custom invite code).	**p0**
Ability to forfeit once in a game.	**p1**
Interactions	
Display the board and all pieces, clearly indicating user colour.	**p0**
Ability to click and drag a piece from the player's respective side upon turn to make a legal move.	**p0**
Highlight all positions on the board to which the selected piece can move.	**p0**
Update the board state when a piece is moved to a valid spot; reset it to its original position for invalid moves.	**p0**
End the game on detection of checkmate or stalemate and display a message indicating the winner.	**p0**
Responsive user interface which adjusts to any device screen size.	**p1**

Highlight the most recent captured piece for each player.	**p1**
Highlight the most recent piece moved.	**p1**
Display all captured pieces for both players along with the respective difference in their total values.	**p1**
Log each move with its notation and timestamp, displayed in reverse chronological order.	**p1**
Timer	
Ability to set a custom timer for local games (can be infinite).	**p3**
Update the timer to countdown based on the active player.	**p0**
Update the respective player's timer state after the turn.	**p0**
End the game and declare a loss for the player with no time remaining.	**p0**
Game Session	
Ensure sessions expire after a period of inactivity, upon logging out or upon forfeit.	**p0**

Backend Requirements	**Priority**
User Account Management	
Ability to login to an existing account (authentication).	**p0**
Game Setup and Logic	
Implement algorithms for valid moves, check detection, checkmate detection, and stalemate detection.	**p0**
Handle all online matchmaking.	**p0**
Implement API for frontend and backend communication.	**p1**
Timer	
Save the timer state after each player's turn.	**p0**
Game Session	
Ensure sessions expire after a period of inactivity, upon logging out or upon forfeit.	**p0**

API Requirements	Priority
Create endpoints for user registration and authentication.	**p0**
Create endpoints for game management (eg. game state, moves history etc.).	**p0**
Create an endpoint to retrieve the timer state.	**p0**

9. Look and Feel Requirements

9.1 Appearance Requirements

Developing Pentachess involves more than just ensuring functionality; it is crucial to focus on user experience as well. Our project will focus on user enjoyment, ensuring they are engaged with the visually appealing details, enhancing their experience with the application and ensuring complete user satisfaction.

Main components:

Board: The board should feature a vibrant yet simple colour palette with shades that are comfortable to view for all users. The pentagon-shaped cells will be colour-coded accurately for easy distinction among categories, e.g. light and dark cells.

Pieces: A consistent colour scheme will be used for each side, with shades that are clearly visible against the chessboard. Each piece will have its unique features ensuring easy distinction of the pieces during play.

User Interface: All elements such as labels and buttons will follow a consistent design system. Buttons, text and all other elements will be placed in common spots with accurate responsiveness indicating to the user that an action has gone through. A reasonable number of colour sets that complement each other will be selected for the UI, ensuring it is easily navigable and not too overwhelming, providing a professional feeling.

9.2 Style Requirements

Objects Scaling: The website shouldn't feature hard-coded components which can cause problems when the window size is modified. All elements will be implemented with window-respective ratios to avoid such issues.

Animations: The animations will be simple and smooth when moving pieces to another cell.

10. Usability and Humanity Requirements

10.1 Ease of Use Requirements

As the software is a game, it must be easy and intuitive. It must be easy to create a game, and all parts of the game (surveying the board, viewing available spaces to move, moving pieces, etc.) must be easy to understand and quick to perform.

10.2 Personalization and Internationalization Requirements

At this point, we do not plan to allow users much personalization. If time allows, we may revisit the various ways that users might want to personalize the game to their style. We may investigate internationalization options, but at this point it is unlikely.

10.3 Learning Requirements
Not only will users come from various backgrounds and familiarity with traditional chess, but they will also need to learn new rules and geometry involved with this variant. Because of this, the game must be easy to learn. Players can be taught directly through a rules page and/or tutorials, and indirectly through the GUI design. Both of these will prove useful to quickly and painlessly get players up to speed with the game's rules.

10.4 Understandability and Politeness Requirements
The application must be presented and explained using terms that members of the general public can understand. Implementation-specific language must not make its way into the user-facing side, and each concept must be explained using language that is both informal and precise.

10.5 Accessibility Requirements
There are a few accessibility concerns that we could reasonably address if given more time: for example, colour blindness and fine motor disability come to mind. Colour blindness could be addressed by providing multiple colour schemes, and fine motor disability could be addressed by giving keyboard-only controls for those who struggle to use a mouse. Unfortunately, this would likely increase the project's scope beyond what is achievable in six months, so we will not be focusing on accessibility.

11. Performance Requirements

11.1 Speed and Latency Requirements
The application must be responsive and provide instant updates when being played locally. As for playing online, the game has a timer for each player, so it is crucial to have the fastest real-time updates on both ends. With this, the goal is to have a maximum of one second to update each move. There should be no noticeable delay when placing the pieces or updating the timer.

11.2 Precision or Accuracy Requirements
Upon selecting a piece, only valid moves must be shown, and the piece should return to its original position if dragged to an invalid cell. Moreover, the timer will be as precise as possible using milliseconds.

11.3 Robustness or Fault-Tolerance Requirements
In the event of disconnection from one side, the game instantly ends and the remaining player gains victory. Allocating some time for the player to reconnect will lead to possible faults and cheats.

11.4 Capacity Requirements
The database should be able to store a reasonable amount of accounts created with information such as name, wins, losses etc. Moreover, the backend should seamlessly handle multiple games running concurrently.

11.5 Scalability or Extensibility Requirements
Any changes to piece movements or board configurations should be easily implemented by changing a few blocks of code.

11.6 Longevity Requirements
There must be a stable hosting provider with guaranteed uptime and potential scalability options. Moreover, we will ensure the use of frameworks such as React.js, which are regularly updated to ensure compatibility with modern browsers and devices. Lastly, backups should be made frequently to ensure user data and stats are saved.

12. Operational and Environmental Requirements

12.1 Expected Physical Environment

- **Device:** Desktop computers/laptops (Windows, MacOS, Linux)
- **Input Devices:** Keyboard and mouse or touchscreen
- **Network:** broadband internet connections (Wi-Fi), preferred low latency for seamless gameplay.

12.2 Requirements for Interfacing with Adjacent Systems

- **Real-Time Gameplay**: Users must be able to play together in real-time using WebSockets (Socket.io)
- **Compatibility with Browsers and Devices**: The application must be accessed via Edge, Chrome, Safari, and Firefox on any computer running a recent version of Linux, MacOS, and Windows
- **Integration of a Database**: Data must be stored in a database (MongoDB or MySQL)
- **Compatibility with Client and Server**: The chosen backend and frontend technologies must minimize latency in communication

12.3 Productization Requirements

- **Scalability**: The backend server must be designed to account for more users in the future
- **Deployment**: The web application must be deployed via a hosting platform with a domain
- **In-Game Documentation**: There must be easy-to-access instructions for users to refer to regarding how to play Pentachess and navigate the website

12.4 Release Requirements

- **Testing**: Manual, unit, integration, and end-to-end testing must be updated and passed for any new addition of code or features
- **Deployment**: Code must be deployed via an established CI/CD pipeline to ensure consistent and safe releases
- **Performance**: Any addition to the real-time gameplay of the application must be tested for response times within the user interface
- **Approval**: Any new features must receive an approval from Dr. Paul Rapoport

13. Maintainability and Support Requirements

13.1 Maintenance Requirements

The application must be constructed in a way that is easy to maintain. Design patterns, code formatting, linting, documentation, etc. must be used so that the discovery of errors, updates to dependencies, and other such causes requiring maintenance can quickly be followed by an update successfully targeting the issue.

13.2 Supportability Requirements

As a web-based application, the software will be supported by any device with a web browser and the necessary peripherals (i.e. mouse and keyboard) to access the game.

13.3 Adaptability Requirements

The application must be constructed such that it is easy to adapt, that is, to add, remove, and change features. As in 13.1, common software practices will be employed to ensure that the software can quickly, easily, and safely be adapted to fit an updated specification.

14. Security Requirements

14.1 Access Requirements

User Authentication:

- Implement secure user registration and login, possibly using OAuth.
- Give the user 5 attempts to login, if unsuccessful, the password must be reset.
- Send an email to the user upon successful login.

14.2 Privacy Requirements

Anonymization:

- Ensure each player's identifiers are anonymized to protect player identities. Only the gamer tag should be shown, no sensitive data such as emails should be retrievable.

Data Collection:

- Store only necessary information from users and provide transparency about its use. Basic requirements such as username, email and password would be ideal.

14.3 Immunity Requirements

Error handling:

- Implement reliable error handling to prevent any known and unknown errors and exceptions.

Backup and Recovery:

- Maintain a record of regular backups for game and user data, ensuring quick recovery from data loss of any sort.

15. Compliance Requirements

15.1 Legal Requirements

We must follow all necessary legal requirements, which in our case, only consist of the following:

- Adhering to Canadian privacy laws; ensuring user information (like user login information) is kept private and secure.
- Ensuring that credit for the game board design, game piece design, and game logic is given to Dr. Paul, our supervisor.

15.2 Standards Compliance Requirements

- Cookies will need to be accepted by our users. We will ensure the GDPR (General Data Protection Regulation) law is followed. Under this law, the purpose and duration of the cookies and how they will be used must be clearly stated to the user. Consent of all cookies will be requested from the user. If the user does not accept, we must not collect.

16. Open Issues

- **Server load management:** It is unclear what the server capacity is and if the servers can handle a high volume of simultaneous players. Stress tests will need to be performed to determine server load management.
- **Multi-timezone Game Sessions:** The impact of game sessions involving users across varying time zones on timer precision, move timestamps, tracking player inactivity and syncing the game state remains unclear.

17. Off-the-Shelf Solutions

17.1 Ready-Made Products

Currently, no off-the-shelf applications or platforms are featuring this specific chess variant. While many chess platforms support a variety of game modes and chess variants, none of the mainstream solutions offer this variant.

17.2 Reusable Components

While no direct implementations of this chess variant exist, several reusable components from existing chess platforms can streamline development. There are a variety of open-source frameworks and libraries widely used for improving developer efficiency and reducing errors:

- **React.js**: A popular library for building interactive, single-page web applications, providing smooth user experiences and real-time data handling.
- **Express.js**: A lightweight web framework for building fast, scalable server-side applications, ideal for handling game logic, user sessions, and API requests.
- **Socket.IO**: Enables real-time, multiplayer functionality through server-client WebSocket connections, essential for live gameplay.
- **Prisma ORM**: A robust tool for managing database interactions, simplifying complex SQL operations, and ensuring data security with features like type safety and SQL injection prevention.

17.3 Products That Can Be Copied

Popular chess platforms such as **Chess.com** and **Lichess.org** offer comprehensive online multiplayer chess experiences. These platforms have the core functionality for standard chess, including user matchmaking, game hosting, and multiplayer interaction. The design and structure of these platforms could serve as a reference for creating a similar experience for this variant.

18. Costs

$10-20 for a potential domain covering the entire year.

19. User Documentation and Training

19.1 User Documentation Requirements

- Users will have access to a detailed rules and regulations page outlining the game's rules and gameplay mechanics.
- A complete user guide will be provided, including instructions on account creation, game setup, and real-time match features.
- An FAQ section will address common questions and troubleshooting steps.

19.2 Training Requirements

No formal training is required, as the game is designed to be intuitive for users familiar with and new to Chess. A section will be dedicated to game rules, allowing the users to familiarize themselves with this chess variant.

20. Waiting Room

With more time, we could leverage the growing trend of integrating artificial intelligence and machine learning into web applications to develop an AI opponent with adjustable difficulty. Additionally, we could implement a leaderboard system to track player rankings and introduce post-game analysis tools that review performance, highlighting mistakes and blunders made during the match.